UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE INGENIERÍA

DIVISIÓN DE INGENIERÍA ELÉCTRICA

INGENIERÍA EN COMPUTACIÓN

LABORATORIO DE COMPUTACIÓN GRÁFICA e INTERACCIÓN HUMANO COMPUTADORA

REPORTE DE PRÁCTICA Nº 03

NOMBRE COMPLETO: CORONEL ENRIQUEZ ALFREDO

Nº de Cuenta: 319028414

GRUPO DE LABORATORIO: 03

GRUPO DE TEORÍA: 07

SEMESTRE 2026-2

FECHA DE ENTREGA LÍMITE: 08/03/2026

CALIFICACIÓN: _____

REPORTE DE PRÁCTICA:

1.- Ejecución de los ejercicios que se dejaron, comentar cada uno y capturas de pantalla de bloques de código generados y de ejecución del programa.

2.- Liste los problemas que tuvo a la hora de hacer estos ejercicios y si los resolvió explicar cómo fue, en caso de error adjuntar captura de pantalla

3.- Conclusión:

   a. Los ejercicios del reporte: Complejidad, Explicación.
   b. Comentarios generales: Faltó explicar a detalle, ir más lento en alguna explicación, otros comentarios y sugerencias para mejorar desarrollo de la práctica
   c. Conclusión

1. Bibliografía en formato APA

PRÁCTICA 03

Modelado geométrico

Comentario:

En esta práctica fue necesario comprender cómo modelar una figura en 3D, analizando su forma y aproximando el espacio que ocupa para poder determinar cómo debía colocarse cada una de sus partes. No solo se trató de dibujar una figura, sino de pensar en la posición, orientación y relación espacial de cada elemento para que el resultado tuviera sentido visualmente.

Para construir el pyraminx se definieron varias funciones de mini pirámides, ya que estas serían las piezas que representarían los colores del cubo tipo Rubik. A partir de ellas se fue armando la estructura general de la figura, utilizando transformaciones como traslación y rotación para acomodarlas en la posición correcta y darles la orientación necesaria dentro del modelo.

Bloques de código:

```
void PiramideMiniVolteada()
{
    unsigned int indices_piramideMinivolt[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramideMinivolt[] = {
        -0.3f,  0.3f,  0.0f,  //0
         0.3f,  0.3f,  0.0f,  //1
         0.0f, -0.3f, -0.3f,  //2
         0.0f,  0.3f, -0.3f   //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramideMinivolt, indices_piramideMinivolt, 12, 12);
    meshList.push_back(obj1);
}

void PiramideMiniVolteada2()
{
    unsigned int indices_piramideMinivolt2[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramideMinivolt2[] = {
        -0.3f,  0.3f,  0.0f,  //0
         0.3f,  0.3f,  0.0f,  //1
         0.0f, -0.3f,  0.0f,  //2
         0.0f,  0.3f, -0.3f   //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramideMinivolt2, indices_piramideMinivolt2, 12, 12);
    meshList.push_back(obj1);
}
```

```cpp
void PiramideMini()
{
    unsigned int indices_piramideMini[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramideMini[] = {
        -0.3f, -0.3f,  0.0f,  //0
         0.3f, -0.3f,  0.0f,  //1
         0.0f,  0.3f, -0.3f,  //2
         0.0f, -0.3f, -0.3f   //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramideMini, indices_piramideMini, 12, 12);
    meshList.push_back(obj1);
}

void PiramideMini2()
{
    unsigned int indices_piramideMini2[] = {
        0,1,2,
        1,3,2,
        3,0,2,
        1,0,3
    };

    GLfloat vertices_piramideMini2[] = {
        -0.3f, -0.3f,  0.3f,  //0
         0.3f, -0.3f,  0.0f,  //1
         0.0f,  0.3f, -0.3f,  //2
         0.0f, -0.3f, -0.3f   //3
    };

    Mesh* obj1 = new Mesh();
    obj1->CreateMesh(vertices_piramideMini2, indices_piramideMini2, 12, 12);
    meshList.push_back(obj1);
}
```

```cpp
int main()
{
    mainWindow = Window(800, 800);
    mainWindow.Initialise();

    //Cilindro y cono reciben resolución (slices, rebanadas) y Radio de circunferencia de la base y tapa
    CrearCubo();                    // indice 0 en MeshList
    CrearPiramideTriangular();      // indice 1 en MeshList
    CrearCilindro(5, 1.0f);         // indice 2 en MeshList
    CrearCono(25, 2.0f);            // indice 3 en MeshList
    CrearPiramideCuadrangular();    // indice 4 en MeshList
    PiramideMini();                 // indice 5 en MeshList
    PiramideMiniVolteada();         // indice 6 en MeshList
    PiramideMini2();                // indice 7 en MeshList
    PiramideMiniVolteada2();        // indice 8 en MeshList
    CreateShaders();

    /*Cámara se usa el comando: glm::lookAt(vector de posición, vector de orientación, vector up));
    En la clase Camera se reciben 5 datos:
    glm::vec3 vector de posición,
    glm::vec3 vector up,
    GlFloat yaw rotación para girar hacia la derecha e izquierda
    GlFloat pitch rotación para inclinar hacia arriba y abajo
    GlFloat velocidad de desplazamiento,
    GlFloat velocidad de vuelta o de giro
    Se usa el Mouse y las teclas WASD y su posición inicial está en 0,0,1 y ve hacia 0,0,-1.
    */

    camera = Camera(glm::vec3(0.0f, 0.0f, 0.0f), glm::vec3(0.0f, 1.0f, 0.0f), -60.0f, 0.0f, 0.3f, 0.3f);


    GLuint uniformProjection = 0;
    GLuint uniformModel = 0;
    GLuint uniformView = 0;
    GLuint uniformColor = 0;

    glm::mat4 projection = glm::perspective(glm::radians(60.0f), mainWindow.getBufferWidth() / mainWindow.getBufferHeight(), 0.1f, 100.0f);
    //glm::mat4 projection = glm::ortho(-1, 1, -1, 1, 1, 10);

    //Loop mientras no se cierra la ventana
    sp.init(); //inicializar esfera
    sp.load();//enviar la esfera al shader
```

```cpp
glm::mat4 model(1.0f);//Inicializar matriz de Modelo 4x4
glm::mat4 baseModel(1.0f);
glm::vec3 color = glm::vec3(0.0f, 0.0f, 0.0f); //inicializar Color para enviar a variable Uniform;

while (!mainWindow.getShouldClose())
{
    GLfloat now = glfwGetTime();
    deltaTime = now - lastTime;
    deltaTime += (now - lastTime) / limitFPS;
    lastTime = now;
    //Recibir eventos del usuario
    glfwPollEvents();
    //Cámara
    camera.keyControl(mainWindow.getsKeys(), deltaTime);
    camera.mouseControl(mainWindow.getXChange(), mainWindow.getYChange());

    //Limpiar la ventana
    glClearColor(1.0f, 1.0f, 1.0f, 1.0f);
    glClear(GL_COLOR_BUFFER_BIT | GL_DEPTH_BUFFER_BIT);
    shaderList[0].useShader();
    uniformModel = shaderList[0].getModelLocation();
    uniformProjection = shaderList[0].getProjectLocation();
    uniformView = shaderList[0].getViewLocation();
    uniformColor = shaderList[0].getColorLocation();

    glUniformMatrix4fv(uniformProjection, 1, GL_FALSE, glm::value_ptr(projection));
    glUniformMatrix4fv(uniformView, 1, GL_FALSE, glm::value_ptr(camera.calculateViewMatrix()));

    // matriz base, siguiendo el estilo del código 2
    baseModel = glm::mat4(1.0f);
    baseModel = glm::translate(baseModel, glm::vec3(0.0f, 0.0f, -4.0f));
    baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotax()), glm::vec3(1.0f, 0.0f, 0.0f));
    baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotay()), glm::vec3(0.0f, 1.0f, 0.0f));
    baseModel = glm::rotate(baseModel, glm::radians(mainWindow.getrotaz()), glm::vec3(0.0f, 0.0f, 1.0f));

    // figura central
    model = baseModel;
    color = glm::vec3(0.0f, 0.0f, 0.0f);
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[1]->RenderMesh();
```

```cpp
// CARA 1

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.00f, -0.30f, -0.03f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.30f, -0.30f, -0.03f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(-0.30f, -0.30f, -0.03f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(-0.15f, -0.24f, -0.05f));
model = glm::rotate(model, glm::radians(305.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[6]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.15f, -0.24f, -0.05f));
model = glm::rotate(model, glm::radians(305.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[6]->RenderMesh();
```

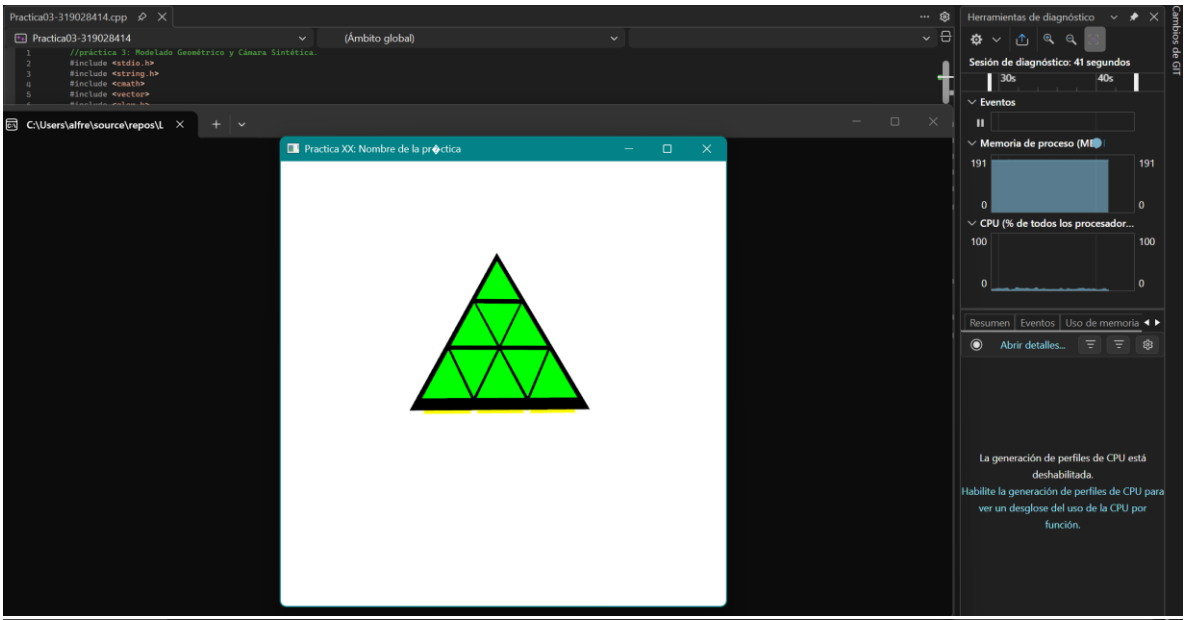
```
model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(-0.15f, 0.00f, -0.18f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.15f, 0.00f, -0.18f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.00f, 0.055f, -0.20f));
model = glm::rotate(model, glm::radians(305.0f), glm::vec3(1.0f, 0.0f, 0.0f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[6]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 1.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.00f, 0.30f, -0.33f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();
```

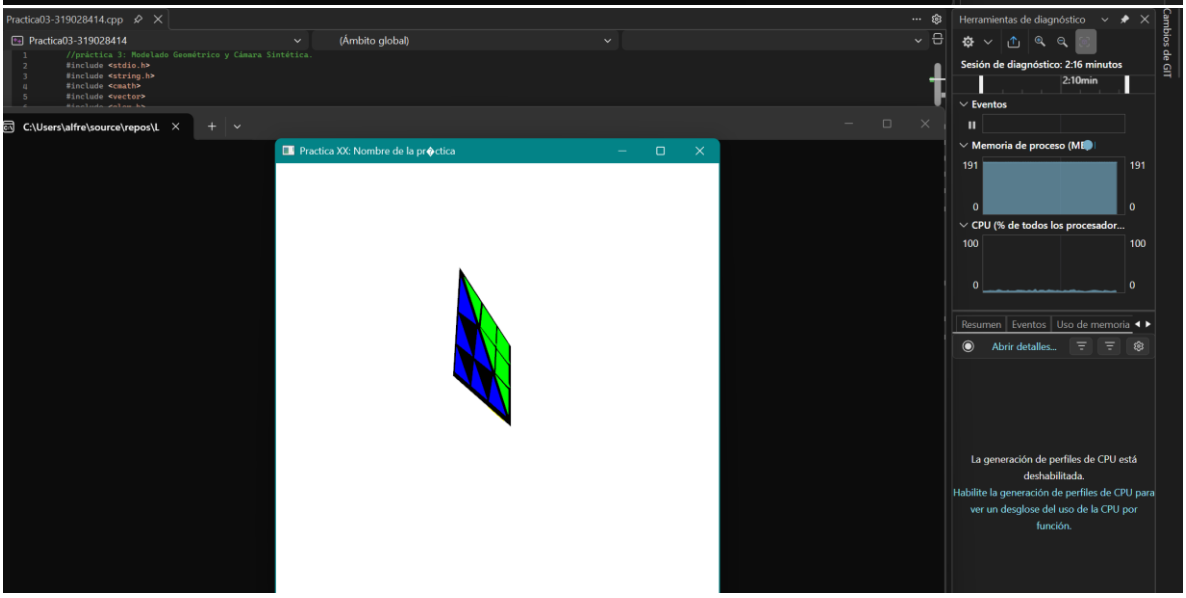
```
//CARA 2

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.32f, -0.30f, -0.05f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.17f, -0.30f, -0.20f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.02f, -0.30f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.02f, 0.00f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.17f, 0.00f, -0.20f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(0.0f, 0.0f, 1.0f);
model = glm::translate(model, glm::vec3(-0.025f, 0.30f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();
```

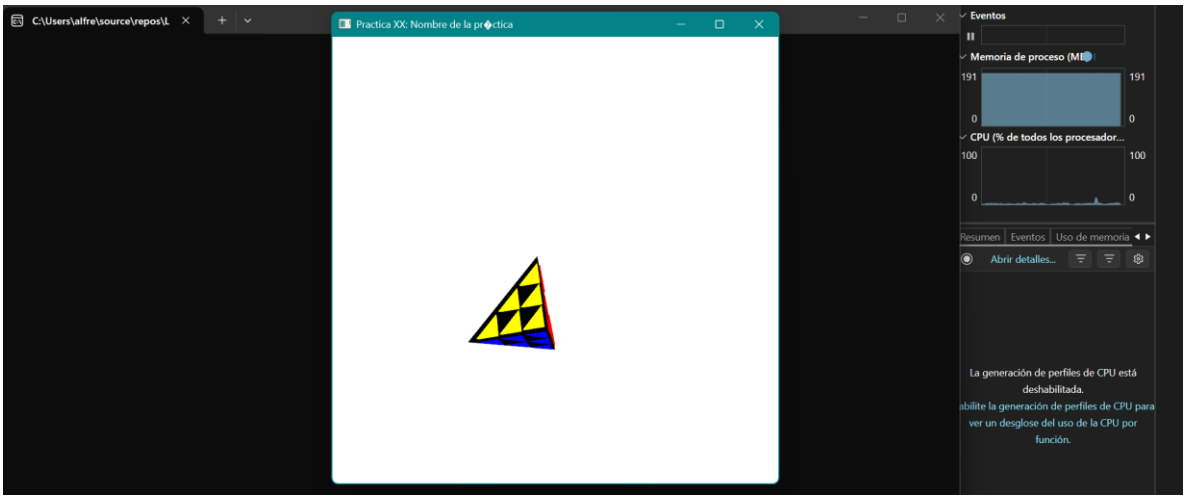
```
// CARA 3

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.33f, -0.30f, -0.05f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.17f, -0.30f, -0.20f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.02f, -0.30f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.04f, 0.00f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.17f, 0.00f, -0.20f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();

model = baseModel;
color = glm::vec3(1.0f, 0.0f, 0.0f);
model = glm::translate(model, glm::vec3(0.03f, 0.30f, -0.35f));
model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
glUniform3fv(uniformColor, 1, glm::value_ptr(color));
meshList[5]->RenderMesh();
```

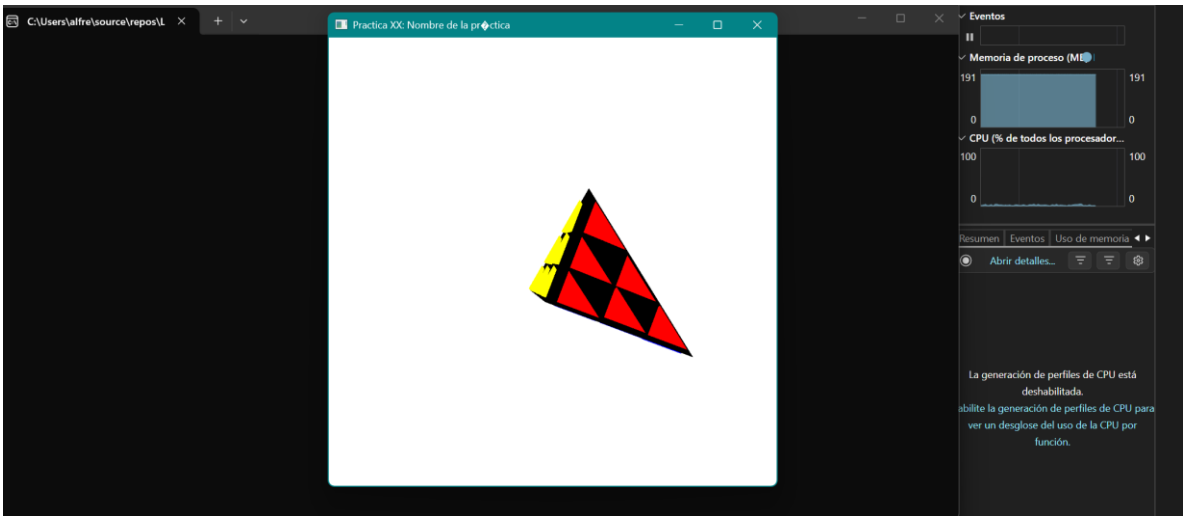
```
    // CARA 4

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(0.00f, -0.39f, -0.05f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(-0.30f, -0.39f, -0.05f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(0.30f, -0.39f, -0.05f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(-0.15f, -0.39f, -0.19f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(0.15f, -0.39f, -0.19f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    model = baseModel;
    color = glm::vec3(1.0f, 1.0f, 0.0f);
    model = glm::translate(model, glm::vec3(0.00f, -0.39f, -0.33f));
    model = glm::scale(model, glm::vec3(0.45f, 0.45f, 0.45f));
    glUniformMatrix4fv(uniformModel, 1, GL_FALSE, glm::value_ptr(model));
    glUniform3fv(uniformColor, 1, glm::value_ptr(color));
    meshList[5]->RenderMesh();

    glUseProgram(0);
    mainWindow.swapBuffers();
}
```

Salida:


Practica03-319028414.cpp
Practica03-319028414
(Ámbito global)
//práctica 3: Modelado Geométrico y Cámara Sintética.
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <vector>
C:\Users\alfre\source\repos\L
Practica XX: Nombre de la pr□ctica
Herramientas de diagnóstico
Sesión de diagnóstico: 41 segundos
30s
40s
Eventos
Memoria de proceso (MB)
191
0
CPU (% de todos los procesador...
100
Resumen
Eventos
Uso de memoria
Abrir detalles...
La generación de perfiles de CPU está deshabilitada.
Habilite la generación de perfiles de CPU para ver un desglose del uso de la CPU por función.
Cambios de GIT


Practica03-319028414.cpp
Practica03-319028414
(Ámbito global)
//práctica 3: Modelado Geométrico y Cámara Sintética.
#include <stdio.h>
#include <string.h>
#include <cmath>
#include <vector>
C:\Users\alfre\source\repos\L
Practica XX: Nombre de la pr□ctica
Herramientas de diagnóstico
Sesión de diagnóstico: 2:16 minutos
2:10min
Eventos
Memoria de proceso (MB)
191
0
CPU (% de todos los procesador...
100
Resumen
Eventos
Uso de memoria
Abrir detalles...
La generación de perfiles de CPU está deshabilitada.
Habilite la generación de perfiles de CPU para ver un desglose del uso de la CPU por función.
Cambios de GIT


C:\Users\alfre\source\repos\L
Practica XX: Nombre de la pr□ctica
Eventos
Memoria de proceso (MB)
191
0
191
0
CPU (% de todos los procesador...
100
0
100
0
Resumen
Eventos
Uso de memoria
Abrir detalles...
La generación de perfiles de CPU está deshabilitada.
abilite la generación de perfiles de CPU para ver un desglose del uso de la CPU por función.


C:\Users\alfre\source\repos\L
Practica XX: Nombre de la pr□ctica
Eventos
Memoria de proceso (MB)
191
0
191
0
CPU (% de todos los procesador...
100
0
100
0
Resumen
Eventos
Uso de memoria
Abrir detalles...
La generación de perfiles de CPU está deshabilitada.
abilite la generación de perfiles de CPU para ver un desglose del uso de la CPU por función.

Conclusión:

Con lo aprendido en esta práctica se logró adquirir una mejor comprensión sobre cómo se realiza el modelado 3D a partir de figuras geométricas simples, así como la manera en que estas deben posicionarse dentro del espacio para formar una estructura más compleja. Aunque fue necesario investigar más de lo esperado para poder avanzar, conceptos como la traslación, la rotación y el acomodo de vértices resultaron fundamentales para entender cómo construir y visualizar correctamente una figura tridimensional. De igual forma, el desarrollo del pyraminx permitió poner en práctica la idea de que no basta con definir una figura, sino que también es importante ajustar su orientación, profundidad y relación con las demás piezas para que el resultado sea el esperado.

Aun así, también quedó claro que este tipo de ejercicios requiere más práctica y una mejor estrategia de trabajo, ya que hubo partes que no se pudieron completar de forma satisfactoria, como el acomodo de algunas pirámides volteadas y ciertos huecos que aparecieron en la figura. Esto hace notar que todavía es necesario seguir reforzando el tema y buscar alternativas que faciliten tanto el proceso de modelado como la corrección de errores visuales.

Bibliografía:

- Khronos Group. (n.d.). *glDrawArrays*. Registry.Khronos.org. Recuperado de: https://registry.khronos.org/OpenGL-Refpages/gl4/html/glDrawArrays.xhtml Consultado el: 06/03/2026.
- https://www.khronos.org/opengl/wiki/Tutorial2:_VAOs,_VBOs,_Vertex_and_Fragment_Shaders_(C_/SDL) Consultado el: 06/03/2026.
- Moreno, F. (n.d.). *Tema 5: Transformaciones geométricas y proyecciones*. Universidad de Huelva. Recuperado de: https://www.uhu.es/francisco.moreno/gii/t/docs/Tema_5.pdf Consultado el: 07/03/2026.